

1 Lesson 02-10-26

In the last lesson we introduced *combinatorial* circuits.

1.1 Datapath components

We now move on to increasingly complex digital building blocks:

- **Combinational:** gates, multiplexors, decoders, ...;
- **Sequential:** basic registers, Finite State Machines, controller;

There is a notion of *control*: *controllers* are good for systems with control inputs/outputs, as opposed to data:

- **Control input:** single bit (or a few), representing environment event or state;
- **Data input:** multiple bits representing single entity.

Our focus will be on **datapath** components at the register-transfer-level, or **RTL**. These are usually networks *controlled* by control networks, which perform functions with both:

- Components to **store** data: registers;
- Components to **transform** data: functional blocks.

1.1.1 Multiplexers

We know from theory that **multiplexers** can implement any combinational logic function (basically a variation on *Shannon's theorem*). This means that they can serve as **LUTs** (*Look Up Tables*) in FPGAs.

A few approaches can be proposed:

1. An $m \times 2^n$ to 1 multiplexer can implement m functions of n inputs. The steps are:
 - Write the truth tables for the m functions;
 - Use the n inputs as selection signals for the multiplexer;
 - Label the m multiplexer outputs with the output functions;
 - Set to 0 or 1 according to each truth table the data inputs of the multiplexer
2. An inverter and an $m \times 2^n$ to 1 multiplexer can implement m functions of n inputs. The steps are:
 - Write the truth tables for the m functions
 - Based on $n - 1$ inputs identify pairs of rows in the tables
 - For each pair and output, identify an elementary function of the n -th input X (that is constant 0 or 1, or X or its negation);
 - Using the first $n - 1$ inputs as indices, assign the data inputs of the multiplexer with the appropriate elementary functions.

The inverter is clearly used to generate X' (negation) in the elementary function, if needed.

1.1.2 Adders

Let's now focus on **adders**. Adders have the (apparently) trivial job of adding together two N bit binary numbers.

Table approach

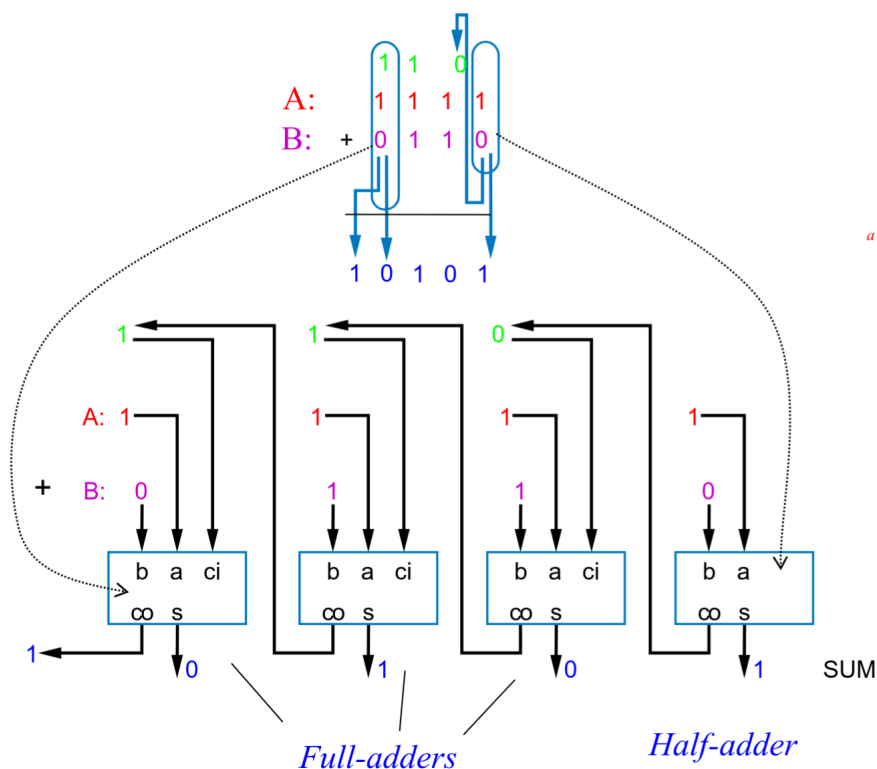
They can be designed using combinational design procedures, but this doesn't work well for typical N (8, 16, 32, ...). The approach scales terribly, as the truth table simply becomes too big to be directly expressed via Shannon expansion.

Hierarchical approach

The idea, then, is to implement in hardware what we usually do to sum numbers by hand:

1. One column at a time;
2. Compute sum of single column;
3. Carry to next column;
4. Repeat until done.

This is trivial (and boring) in hardware, with ripple carry:



Two components are used in the figure: an *half adder*, and a set of *adders*. Using half-adder and full-adders, we can build adder that adds like we would by hand:

- The component is called a *ripple-carry adder*;
- 4-bit adder shown: Adds two 4-bit numbers, generates 5-bit output;
- 5-bit output can be considered 4-bit "sum" plus 1-bit "carry out";

- We can easily build any size adder.

Half adder

The **half adder** is the most trivial component: it takes 2 bits (a and b) and returns:

- s , the LSB of the sum of the bits;
- c_{out} , the MSB of the sum of the bits (they can evaluate to binary 2, which is 10).

The half adder is easy to implement both via LUTs, but also via discrete logic gates:

$$s = a \otimes b, \quad c_{out} = a \cdot b$$

where $\otimes$ is notation for XOR.

Adder

The **adder** (or **full adder**) is the slightly more complicated variant of the half adder: it sums 3 inputs instead of 2. This is needed to correctly implement ripple carries, as we have the two digits at column i plus the ripple carry from column $i - 1$ (which is always zero for the first digit, therefore justifying an half adder).

The full adder takes 3 bits (a , b and c_{in}) and returns:

- s , the LSB of the sum of the bits;
- c_{out} , the MSB of the sum of the bits (they can evaluate to binary 2, which is 10, or even binary 3, which is 11).

The full adder is easy to implement both via LUTs, but also via discrete logic gates or hierarchically, with half adders:

$$s = a \otimes b \otimes c_{in}, \quad c_{out} = ab + ac_{in} + bc_{in} = (a \otimes b) \cdot c_{in} + a \cdot b$$

Note that the XOR operator (again, $\otimes$) acts as a *disparity checker* operator: for $\otimes_i^n \alpha_i$, if an odd number of α_i inputs are high, the XOR operator returns true.

Note that using a full-adder instead of half-adder for first bit, we can include a “carry in” bit in the addition circuit. This is useful later when we connect smaller adders to form bigger adders.

Adder timing characteristics

Ripple carry adders have specific timing behaviour: operations have to propagate (that is, *ripple*) through all adder modules for the results to be correct. This gives non-ideal timing characteristics (more bits mean significantly more delay, with linear scaling).

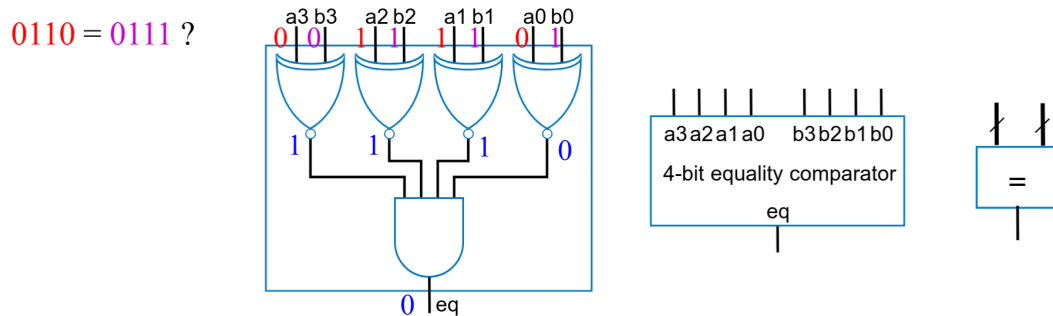
1.1.3 Comparators

We would now want to **compare** data, that is build a device which outputs 1 if two N-bit numbers are equal.

Equality comparator

If we are taking the **equality** of numbers, this is trivial. This is easy to achieve via XORs. For instance, for 4 bits:

$$eq = (a_3b_3 + a'_3b'_3) \cdot (a_2b_2 + a'_2b'_2) \cdot (a_1b_1 + a'_1b'_1) \cdot (a_0b_0 + a'_0b'_0)$$

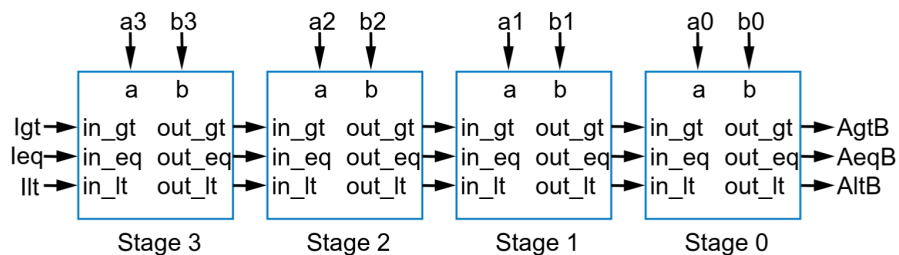


Magnitude comparator

We might also want a *magnitude*, comparator. We consider comparing by hand:

- First compare a_3 and b_3 . If equal, compare a_2 and b_2 . And so on;
- Stop if comparison not equal (the two bits are 0 and 1, or 1 and 0): whichever of A or B has the 1 is thus greater.
- If we never see an unequal bit pair, then $A=B$.

This gives us the following circuit:



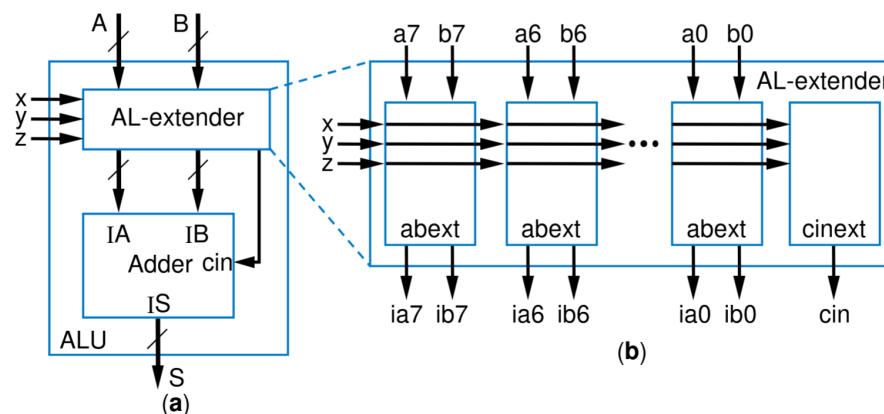
Of course, we do need a single digit comparator. Thankfully, this is easy to accomplish.

1.1.4 Arithmetic Logic Units

An **ALU** (*Arithmetic Logic Unit*) is a component that can perform various arithmetic (add, subtract, increment, etc.) and logic (AND, OR, etc.) operations, based on control inputs.

Inputs			Operation	Sample output if A=00001111, B=00000101
x	y	z		
0	0	0	$S = A + B$	S=00010100
0	0	1	$S = A - B$	S=00001010
0	1	0	$S = A + 1$	S=00010000
0	1	1	$S = A$	S=00001111
1	0	0	$S = A \text{ AND } B$ (bitwise AND)	S=00000101
1	0	1	$S = A \text{ OR } B$ (bitwise OR)	S=00001111
1	1	0	$S = A \text{ XOR } B$ (bitwise XOR)	S=00001010
1	1	1	$S = \text{NOT } A$ (bitwise complement)	S=11110000

- The simplest way to implement an ALU is using separate components for each operation, and muxes. This, though, requires too many wires, and also wastes power computing operations when we only actually use one result at given time.
- Because of this, ALU designs don't just have separate components, multiplexed together. Most ALU designs use a *single adder*, plus logic in front of adder's A and B inputs. This logic is called an **arithmetic-logic extender**: the extender modifies A and B inputs so desired operation appears at output of the adder.



1.2 VHDL delay models

There are 3 ways to model delays in VHDL:

1. **Inertial delay** deletes pulses that are less than a threshold long. This is suitable for modeling delays through devices such as gate. The syntax is:

```
Y1 <= not(A and B) after 7ns;
```
2. **Transport delay** transports all pulses, i.e., all input events are propagated to output signals. It models delays through devices with very small inertia, e.g, wires: seldom used in hardware design. The syntax is:

```
Y2 <= not(A and B) transport after 7ns;
```
3. **Delta delay** means no propagation delays are specified. Infinitesimally small delay is automatically inserted by the simulator to preserve correct ordering of events (projected values become current values).